

1. 시스템콜 작동 과정 분석

리눅스에서 `write()` 시스템 콜은 사용자 프로그램이 커널에게 출력 작업을 요청하는 과정이다. 사용자 프로그램은 커널 내부 함수에 직접 접근할 수 없기 때문에, 정해진 시스템 콜 인터페이스를 통해 요청을 전달한다.

먼저 사용자 프로그램이 `write(fd, buf, count)`를 호출하면, C 라이브러리 래퍼 함수가 호출된다. 이 래퍼는 `write`에 해당하는 시스템 콜 번호와 인자들을 준비한 뒤 `syscall` 명령어를 실행한다. `syscall` 명령어가 실행되면 CPU는 사용자 모드에서 커널 모드로 전환된다.

커널에 진입한 뒤에는 전달된 시스템 콜 번호를 기준으로 시스템 콜 테이블을 확인한다. 시스템 콜 테이블은 각 시스템 콜 번호를 실제 커널 핸들러 함수와 연결해 주는 표이다. `write`의 경우 해당 번호를 통해 `sys_write` 계열의 커널 핸들러 함수가 호출된다.

커널 핸들러 함수는 파일 디스크립터, 버퍼 주소, 바이트 수를 확인하고 실제 쓰기 작업을 수행한다. 작업이 끝나면 커널은 성공 시 작성된 바이트 수를 반환하고, 실패 시 오류 값을 반환한다. 이후 `sysret`과 같은 복귀 과정을 통해 다시 사용자 모드로 돌아간다.

전체 흐름은 `write()` 호출 → C 라이브러리 래퍼 → `syscall` 명령어 → 시스템 콜 번호 확인 → 시스템 콜 테이블 참조 → 커널 핸들러 실행 → 결과 반환 → `sysret`을 통한 사용자 모드 복귀 순서로 진행된다. 이 구조를 통해 사용자 프로그램은 커널 기능을 안전하게 사용할 수 있다.

2. 시스템콜 생성 및 커널 컴파일 실습

`syscall_64.tbl` 파일에 새로운 시스템 콜 번호와 함수 이름을 등록하였다.

```

456     common    futex_requeue      sys_futex_requeue
457     common    statmount          sys_statmount
458     common    listmount          sys_listmount
459     common    lsm_get_self_attr   sys_lsm_get_self_attr
460     common    lsm_set_self_attr   sys_lsm_set_self_attr
461     common    lsm_list_modules    sys_lsm_list_modules
462     common    print_student_name  sys_print_student_name
463     common    print_student_id    sys_print_student_id
464     common    print_student_info  sys_print_student_info

#
# Due to a historical design error, certain syscalls are numbered differently
# in x32 as compared to native x86_64. These syscalls have numbers 512-547

```

462 번은 sys_print_student_name, 463 번은 sys_print_student_id, 464 번은 sys_print_student_info 함수와 연결하였다. 이 작업은 사용자 프로그램에서 syscall()로 해당 번호를 호출했을 때, 커널이 실행할 함수를 찾을 수 있도록 하기 위해 필요하다.

syscalls.h 파일에 새 시스템 콜 함수들의 프로토타입을 추가하였다.

```

        int __user *optlen);
int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
        int optlen);
asmlinkage long sys_print_student_name(char __user *name);
asmlinkage long sys_print_student_id(char __user *id);
asmlinkage long sys_print_student_info(char __user *school, char __user *major);
#endif

```

이 헤더 파일 수정은 커널 컴파일 과정에서 새로 만든 시스템 콜 함수들의 반환형, 함수 이름, 인자 형식을 미리 알려 주기 위해 필요하다. 이를 통해 커널이 해당 함수들을 올바르게 인식하고 연결할 수 있다.

kernel/Makefile 의 obj-y 항목에 new_syscall.o 를 추가하였다.

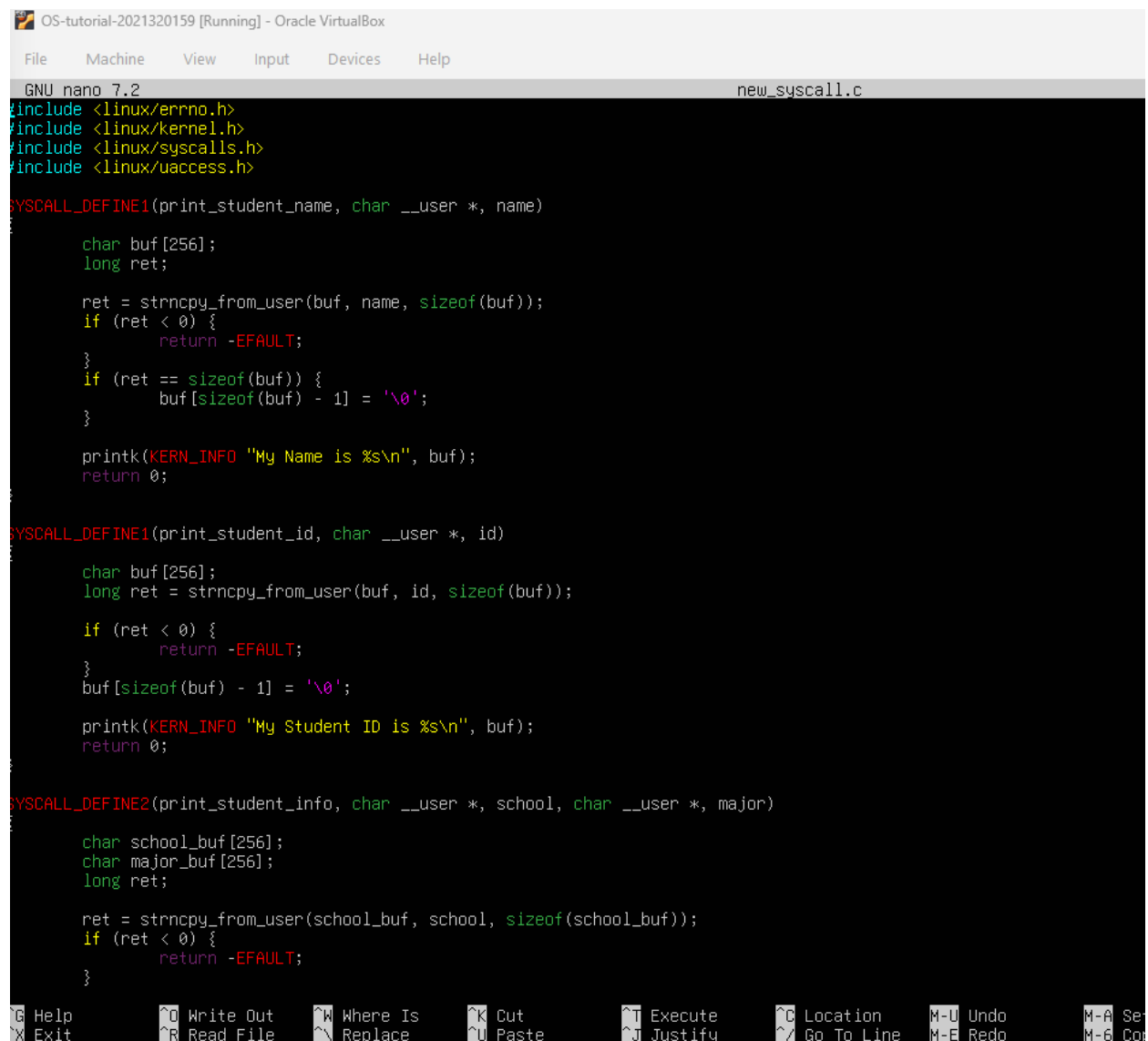
```

obj-y    = fork.o exec_domain.o panic.o \
          cpu.o exit.o softirq.o resource.o \
          sysctl.o capability.o ptrace.o user.o \
          signal.o sys.o umh.o workqueue.o pid.o task_work.o \
          extable.o params.o \
          kthread.o sys_ni.o nsproxy.o \
          notifier.o ksysfs.o cred.o reboot.o \
          async.o range.o smpboot.o ucount.o regset.o ksyms_common.o new_syscall.o

```

이 수정은 새로 작성한 new_syscall.c 파일이 커널 빌드 과정에 포함되도록 하기 위해 필요하다. 이를 추가하지 않으면 소스 파일을 작성해도 컴파일되지 않기 때문에, 시스템 콜 함수가 커널에 포함되지 않는다.

new_syscall.c 파일에는 세 개의 새로운 시스템 콜 함수를 구현하였다.



```
OS-tutorial-2021320159 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 7.2 new_syscall.c
#include <linux/errno.h>
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE1(print_student_name, char __user *, name)
{
    char buf[256];
    long ret;

    ret = strncpy_from_user(buf, name, sizeof(buf));
    if (ret < 0) {
        return -EFAULT;
    }
    if (ret == sizeof(buf)) {
        buf[sizeof(buf) - 1] = '\0';
    }

    printk(KERN_INFO "My Name is %s\n", buf);
    return 0;
}

SYSCALL_DEFINE1(print_student_id, char __user *, id)
{
    char buf[256];
    long ret = strncpy_from_user(buf, id, sizeof(buf));

    if (ret < 0) {
        return -EFAULT;
    }
    buf[sizeof(buf) - 1] = '\0';

    printk(KERN_INFO "My Student ID is %s\n", buf);
    return 0;
}

SYSCALL_DEFINE2(print_student_info, char __user *, school, char __user *, major)
{
    char school_buf[256];
    char major_buf[256];
    long ret;

    ret = strncpy_from_user(school_buf, school, sizeof(school_buf));
    if (ret < 0) {
        return -EFAULT;
    }
}
```

sys_print_student_name 은 사용자 공간에서 이름 문자열을 인자로 받아 strncpy_from_user()로 커널 공간에 복사한 뒤, printk()를 사용하여 이름을 커널 로그에 출력한다.

sys_print_student_id 는 학번 문자열을 인자로 받아 커널 공간에 복사한 뒤, printk()로 학번을 출력한다.

sys_print_student_info 는 학교 이름과 전공 문자열을 인자로 받아 각각 커널 공간에 복사한 뒤, printk()로 학교와 전공 정보를 출력한다.

각 함수는 사용자 공간의 문자열을 안전하게 가져오기 위해 strncpy_from_user()를 사용하였고, 복사에 실패하면 -EFAULT 를 반환하도록 구현하였다.

커널 빌드 과정에서 인증서 관련 오류를 방지하기 위해 .config 파일의 인증 키 설정을 수정하였다.

```
# Certificates for signature checking
#
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS=""
# CONFIG_SYSTEM_EXTRA_CERTIFICATE is not set
# CONFIG_SECONDARY_TRUSTED_KEYRING is not set
# CONFIG_SYSTEM_BLACKLIST_KEYRING is not set
# end of Certificates for signature checking
```

실습 안내에는 CONFIG_SYSTEM_REVOCATION_KEYS 도 함께 수정하도록 되어 있었지만, 현재 .config 파일에서는 해당 항목이 따로 존재하지 않았다. 따라서 확인 가능한 CONFIG_SYSTEM_TRUSTED_KEYS 항목을 빈 문자열로 변경하였다. 이를 통해 커널 빌드 시 별도의 인증 키 파일을 참조하지 않도록 설정하였다.

커널 컴파일과 설치 후 uname -r 명령어를 사용하여 현재 실행 중인 커널 버전을 확인하였다.

```
System load:          0.41
Usage of /:           33.1% of 32.18GB
Memory usage:         3%
Swap usage:           0%
Processes:            103
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fed0:3c3c

Expanded Security Maintenance for Applications is not enabled.

22 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

[sudo] password for jwk:
/sbin/mount.vboxsf: mounting failed with the error: No such device
/sbin/mount.vboxsf: mounting failed with the error: No such device
jwk@2021320159:~$ uname -r
5.8.12
jwk@2021320159:~$
```

이 결과는 시스템이 새로 빌드한 6.8.12 커널로 정상 부팅되었음을 의미한다. 따라서 앞에서 수정한 시스템 콜 관련 코드가 포함된 커널이 현재 실행 중인 상태임을 확인할 수 있다.

사용자 프로그램인 `call_new_syscall.c` 에서 `syscall()` 함수를 사용하여 새로 만든 시스템 콜을 직접 호출하였다.

```
GNU nano 7.2                                call_new_syscall.c
#include <unistd.h>

int main(void)
{
    char name[] = "Junwoo Kim";
    char id[] = "2021320159";
    char school[] = "Korea University";
    char major[] = "Computer Science and Engineering";
    syscall(462, name);
    syscall(463, id);
    syscall(464, school, major);
    return 0;
}

jwk@2021320159:~/workspace$
```

462 번 시스템 콜에는 이름 문자열을 전달하였고, 463 번 시스템 콜에는 학번 문자열을 전달하였다. 464 번 시스템 콜에는 학교 이름과 전공 문자열을 함께 전달하였다. 이를 통해 사용자 공간의 프로그램에서 커널에 등록한 새 시스템 콜 함수들을 호출할 수 있다.

`call_new_syscall.c` 를 컴파일하고 실행한 뒤, `dmesg` 명령어를 사용하여 커널 로그를 확인하였다.

```

/w.
[ 4.325301] Adding 4008956k swap on /swap.img. Priority:-2 extents:14 across
:17574912k
[ 4.340177] systemd-journald[291]: Received client request to flush runtime j
ournal.
[ 4.387245] systemd-journald[291]: /var/log/journal/1bc0e5c393ac439da06f68bea
4842216/system.journal: Journal file uses a different sequence number ID, rotati
ng.
[ 4.388709] systemd-journald[291]: Rotating system journal.
[ 5.167903] EXT4-fs (sda2): mounted filesystem c2948731-542b-42f1-86e5-af94af
05acc9 r/w with ordered data mode. Quota mode: none.
[ 5.344969] e1000: enp0s3 NIC Link is Up 1000 Mbps Full Duplex, Flow Control:
RX
[ 6.060919] loop0: detected capacity change from 0 to 8
[ 6.064859] e2scrub_all (534) used greatest stack depth: 12768 bytes left
[ 17.699177] systemd-journald[291]: /var/log/journal/1bc0e5c393ac439da06f68bea
4842216/user-1000.journal: Journal file uses a different sequence number ID, rot
ating.
[ 499.328459] clocksource: Long readout interval, skipping watchdog check: cs_n
sec: 1669352539 wd_nsec: 1669352228
[ 553.613809] My Name is Junwoo Kim
[ 553.613815] My Student ID is 2021320159
[ 553.613819] I go to Korea University
[ 553.613821] I major in Computer Science and Engineering
jwk@2021320159:~/workspace$ _

```

위 메시지들은 new_syscall.c 에서 printk()로 출력하도록 구현한 내용이다. 따라서 syscall(462), syscall(463), syscall(464)가 정상적으로 호출되었고, 커널 내부의 시스템 콜 함수들이 성공적으로 실행되었음을 확인할 수 있다.